Compte rendu – TP2 d'Ordonnancement Temps Réel

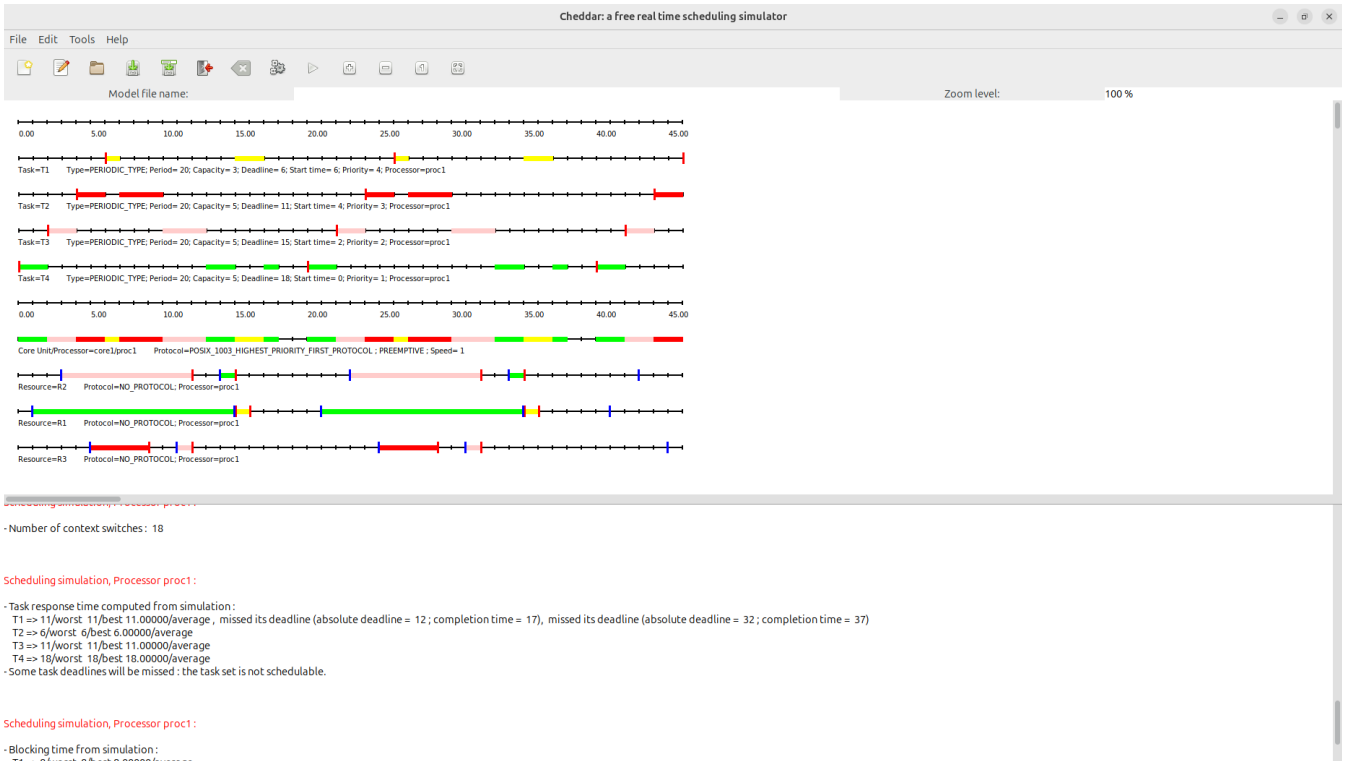
Bongiovanni Arthur
3A SN - (E)

Exercice 1

1. Sans protocole

Sans protocole de synchronisation, les tâches peuvent entrer en section critique indépendamment de leur priorité.
Cela provoque des blocages transverses :
une tâche de faible priorité peut posséder une ressource dont une tâche plus prioritaire a besoin, entraînant un effet d'inversion de priorité.

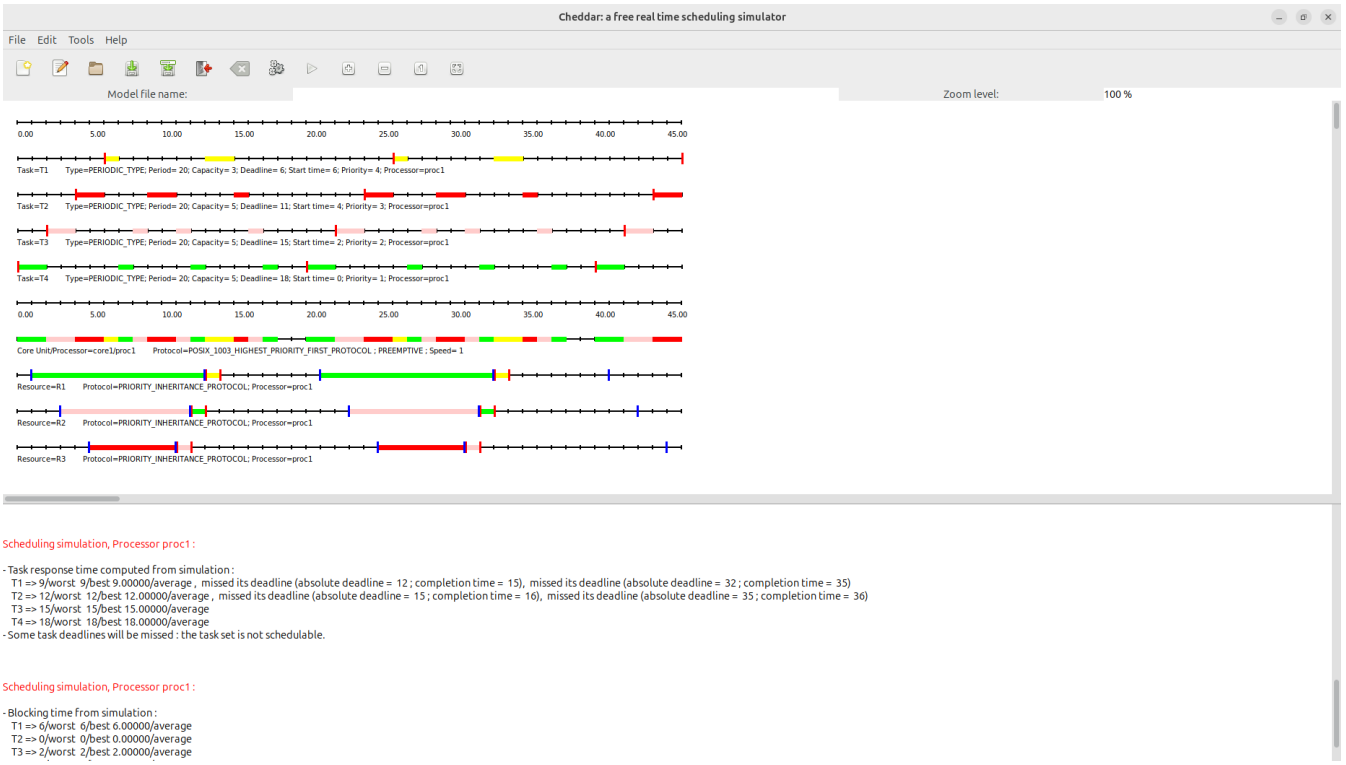
Dans la simulation, T1 rate sa deadline car T4 (moins prioritaire) garde la ressource R1 trop longtemps.



2. Priority Inheritance Protocol (PIP)

Le Priority Inheritance Protocol améliore la situation en limitant la durée des blocages. Lorsqu'une tâche basse priorité bloque une tâche plus prioritaire, elle hérite temporairement de sa priorité jusqu'à la libération de la ressource.

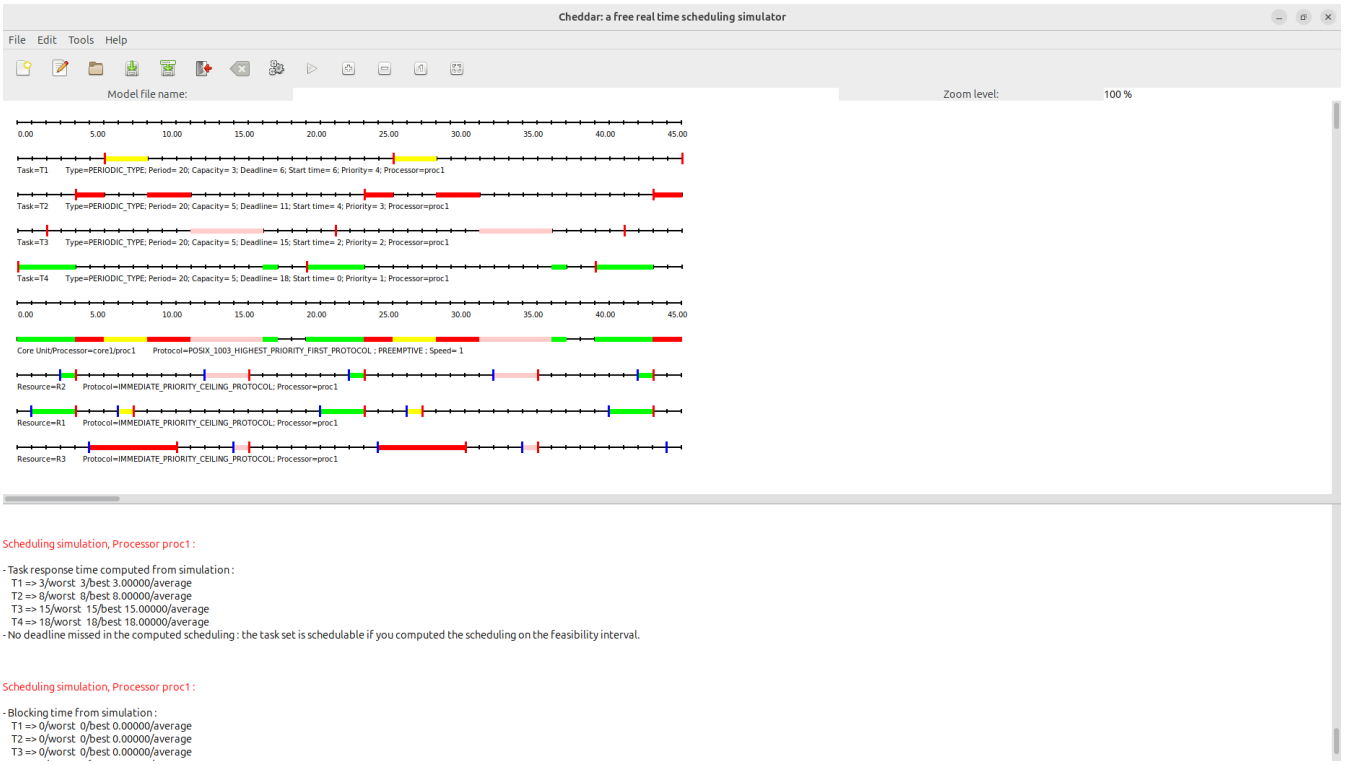
Ce mécanisme réduit la durée du blocage mais n'élimine pas tous les scénarios pathologiques : une chaîne de blocages peut encore se produire si plusieurs ressources sont imbriquées ou partagées entre plusieurs niveaux de priorité.



3. Stack-Based / Immediate Ceiling Inheritance (ICI)

Le protocole Stack-Based / Immediate Ceiling Inheritance empêche une tâche d'entrer en section critique si une ressource déjà allouée possède un plafond supérieur ou égal à sa priorité.
Ce mécanisme garantit qu'aucune tâche basse priorité ne peut bloquer une tâche plus prioritaire indirectement.

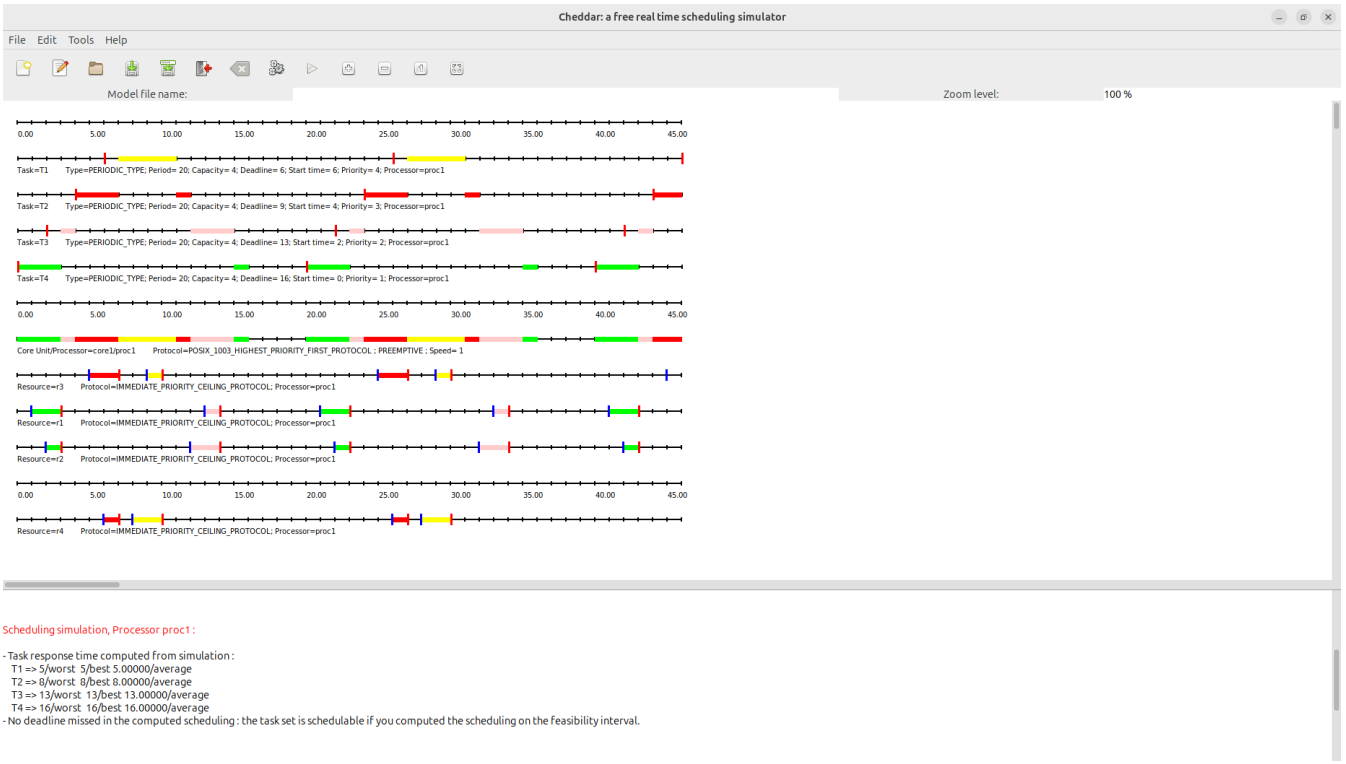
Résultat : toutes les deadlines sont respectées, car les blocages imprévisibles sont éliminés.



Exercice 2 — Application du protocole ICI

L'application du protocole Stack-Based / Immediate Ceiling Inheritance empêche les blocages prolongés.
Une tâche à faible priorité ne peut pas verrouiller une ressource susceptible de bloquer une tâche plus prioritaire.

Comme dans l'exercice 1 (partie 3), la simulation montre que le système est ordonnançable sous ce protocole.



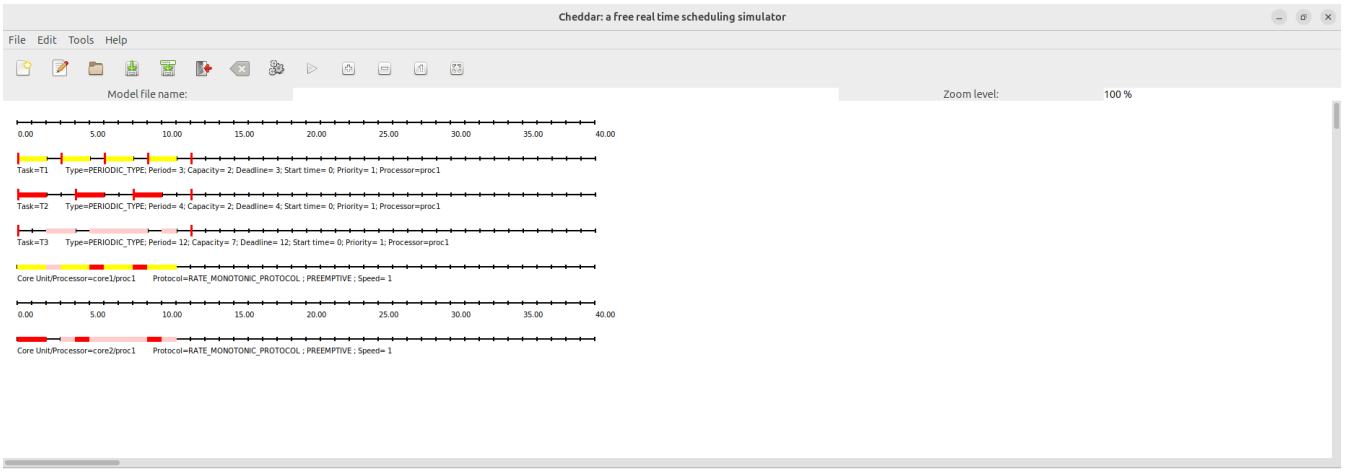
Exercice 3 — Ordonnancement global sur 2 cœurs

> Principe : toutes les tâches appartiennent à une même file d'attente globale.
> À chaque instant, les deux processeurs exécutent les deux tâches prêtes les plus prioritaires.
> Les tâches peuvent migrer entre cœurs d'une période à l'autre.

1. Cas 1 — Système ordonnançable

Dans ce premier cas, la demande processeur cumulée reste inférieure à la capacité totale (2 cœurs).
Aucune coïncidence d'activations ne provoque de surcharge instantanée.

Toutes les tâches respectent leur deadline : le système est ordonnançable.



Time-Unit migration

Scheduling simulation, Processor proc1 :
- Number of preemptions : 10

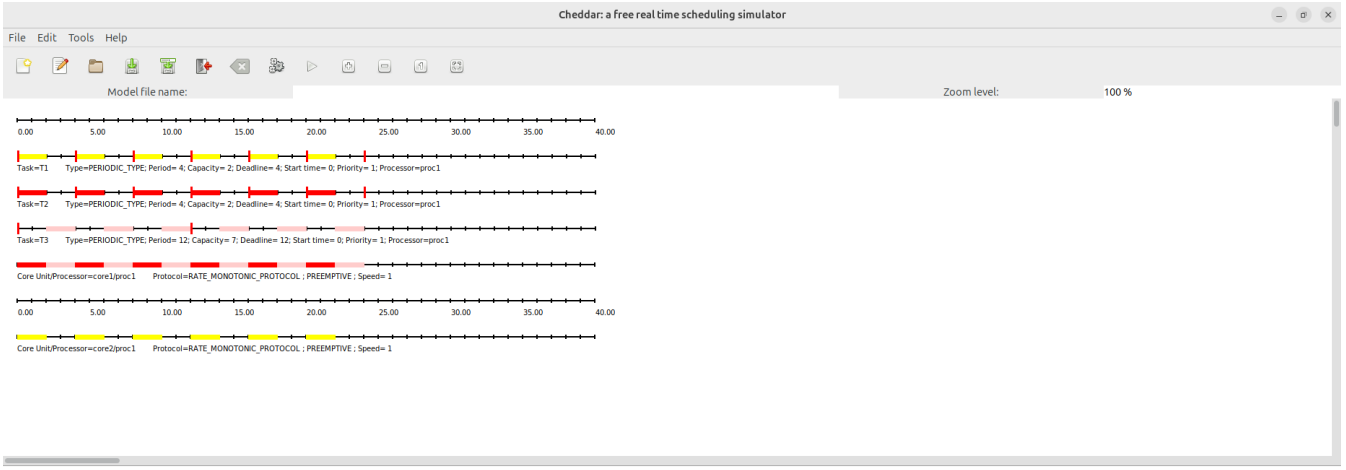
Scheduling simulation, Processor proc1 :
- Number of context switches : 19

Scheduling simulation, Processor proc1 :
- Task response time computed from simulation :
  T1 => 2/worst 2/best 2.00000/average
  T2 => 2/worst 2/best 2.00000/average
  T3 => 11/worst 11/best 11.00000/average
- No deadline missed in the computed scheduling : the task set is schedulable if you computed the scheduling on the feasibility interval.

2. Cas 2 — Système non ordonnançable

Dans cette configuration, la somme des utilisations reste ≤ 2, mais les activations sont synchronisées :
plusieurs tâches de haute priorité arrivent simultanément, saturant les deux cœurs pendant une fenêtre critique.

La tâche T3 ne peut pas obtenir assez de temps CPU avant sa deadline -> deadline manquée.



plus de ressource mais toujours en parallèle donc on ne peut utiliser qu'un seul coeur à la fois (moins de ressource utilisable par rapport au scénraio 1)

Scheduling simulation, Processor proc1 :
- Number of preemptions : 11

Scheduling simulation, Processor proc1 :
- Number of context switches : 29

Scheduling simulation, Processor proc1 :

- Task response time computed from simulation :
  T1 => 2/worst 2/best 2.00000/average
  T2 => 2/worst 2/best 2.00000/average
  T3 => 15/worst 15/best 15.00000/average , missed its deadline (absolute deadline = 12 ; completion time = 15)
- Some task deadlines will be missed : the task set is not schedulable.

3. Interprétation

- Le cas 2 montre que, dans un ordonnancement global multiprocesseur, le test :
$$U_{\text{total}} \leq m$$
(avec m le nombre de cœurs) n'est pas suffisant pour garantir la faisabilité.
- L'instant critique correspond à la coïncidence des arrivées de plusieurs tâches prioritaires, ce qui retarde les autres.
- Le Global Rate Monotonic n'est pas optimal : même avec des processeurs disponibles « en moyenne », certaines tâches peuvent rater leur deadline.

En pratique, on préfère souvent les approches partitionnées (chaque processeur possède son propre ordonnanceur local RM ou EDF) pour plus de prédictibilité.

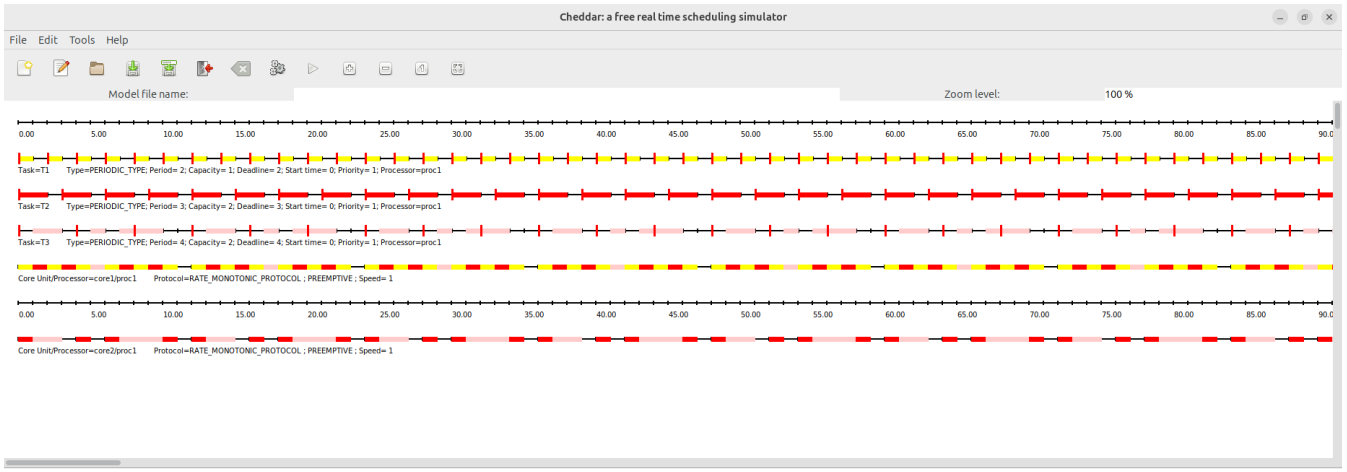
Exercice 4 — Détermination du pire cas d'activation

Ici, on cherche à identifier le moment où la demande cumulée est maximale pour une tâche donnée (T3).
La simulation montre que le pire cas n'apparaît pas à t = 0, mais lors du second réveil de T3 (t = 4), puis se reproduit toutes les trois périodes.

Explication

- Au démarrage (t=0), les tâches sont déphasées de manière favorable.
- À t=4, plusieurs activations coïncident : les tâches de plus haute priorité monopolisent les processeurs.
- T3 est retardée et subit un temps de réponse maximal à ce moment-là.

Cet instant correspond au pire scénario d'interférence, appelé instant critique, qui se répète de façon périodique.
Ce type d'analyse est essentiel pour dimensionner les deadlines ou vérifier la robustesse de l'ordonnanceur.



Scheduling simulation, Processor proc1 :
- Number of preemptions : 54

Scheduling simulation, Processor proc1 :
- Number of context switches : 323

Scheduling simulation, Processor proc1 :

- Task response time computed from simulation :
  T1 => 1/worst 1/best 1.00000/average
  T2 => 2/worst 2/best 2.00000/average
  T3 => 4/worst 2/best 3.00000/average
- No deadline missed in the computed scheduling : the task set is schedulable if you computed the scheduling on the feasibility interval.